



Seguridad informática aplicada a Entornos industriales

**Diseño e implementación de una arquitectura
DevSecOps basada en GitOps:
Aseguramiento de la cadena de suministro en
Kubernetes mediante scanners, ArgoCD y
Kyverno**

Memòria

**Ziad El Karrabi El Hanafi
Tutor: Josep Andreu**

2026

Índice general

Índice de figuras	III
Índice de cuadros	V
1. Introducción	1
2. Marco teórico y análisis de referentes	3
2.1. Evolución del paradigma DevOps a DevSecOps	3
2.1.1. Limitaciones de los modelos tradicionales	3
2.1.2. Integración de la seguridad en el ciclo de vida del software	3
2.1.3. Nacimiento del concepto DevSecOps	4
2.2. Kubernetes como plataforma de ejecución	4
2.2.1. Arquitectura de Kubernetes	4
2.2.2. Componentes principales del clúster	4
2.2.3. Gestión de red y Network Policies	5
2.3. Policy-as-Code y Admission Controllers	5
2.3.1. Funcionamiento de los Admission Controllers	5
2.3.2. Validación vs mutación de recursos	5
2.3.3. Importancia de la seguridad declarativa	6
2.4. Herramientas y tecnologías utilizadas	6
2.4.1. Kyverno	6
2.4.2. OPA Gatekeeper	6
2.4.3. ArgoCD y GitOps	7
2.4.4. GitHub Actions	7
2.4.5. Trivy y Sysdig Secure	7
2.5. Comparativa teórica Kyverno vs OPA	7
3. Objetivos y alcance	9
3.1. Objetivo general	9
3.2. Objetivos específicos	9
3.3. Alcance del proyecto	10
Bibliografía	11

Índice de figuras

1.1. Arquitectura general del entorno DevSecOps	2
---	---

Índice de cuadros

2.1. Comparativa entre Kyverno y OPA Gatekeeper	8
---	---

1 Introducción

La adopción de arquitecturas basadas en contenedores y orquestadores como Kubernetes ha transformado la forma en que se desarrollan y despliegan aplicaciones en entornos cloud y corporativos. Este paradigma permite automatizar la gestión del ciclo de vida de las aplicaciones, mejorar la escalabilidad y facilitar la integración de prácticas de desarrollo modernas. Sin embargo, esta evolución tecnológica también introduce nuevos desafíos en materia de seguridad, especialmente en entornos donde los despliegues se realizan de forma automatizada y continua.

En este contexto surge el enfoque DevSecOps, cuyo objetivo es integrar la seguridad dentro del propio ciclo de vida del desarrollo de software, en lugar de tratarla como una fase posterior al despliegue. Este modelo promueve la automatización de controles de seguridad dentro de los pipelines de integración y despliegue continuo, permitiendo detectar vulnerabilidades, aplicar políticas de seguridad y garantizar configuraciones seguras desde las primeras fases del desarrollo.

El presente Trabajo de Fin de Máster se centra en la implementación de un entorno DevSecOps completo sobre Kubernetes, integrando herramientas de Integración Continua (GitHub Actions) con análisis automatizado de vulnerabilidades mediante herramientas como Trivy y Sysdig Secure. Asimismo, se implementará un modelo de Despliegue Continuo basado en GitOps utilizando ArgoCD, permitiendo gestionar la infraestructura y los despliegues de forma declarativa a través de repositorios Git.

Adicionalmente, se incorporará el uso de Kyverno como motor de políticas de seguridad basado en Policy-as-Code, permitiendo aplicar controles automáticos sobre los manifiestos de Kubernetes durante el proceso de despliegue.

El núcleo del proyecto se basa en el uso de Admission Controllers, concretamente Kyverno, para aplicar políticas de seguridad de forma declarativa dentro del clúster de Kubernetes. A diferencia de los enfoques tradicionales que únicamente bloquean configuraciones inseguras, este trabajo explorará el uso de políticas de tipo mutating, capaces de corregir automáticamente configuraciones inseguras en los manifiestos YAML en tiempo de despliegue.

Finalmente, el proyecto incluirá un análisis comparativo entre Kyverno y OPA Gatekeeper, evaluando sus capacidades en la implementación de políticas de seguridad en Kubernetes, su facilidad de uso, curva de aprendizaje y eficacia en entornos de producción.

La Figura 1.1 muestra una visión general de la arquitectura DevSecOps implementada en este trabajo, donde se integran los procesos de desarrollo, análisis de seguridad, despliegue continuo y aplicación de políticas dentro de un entorno Kubernetes.

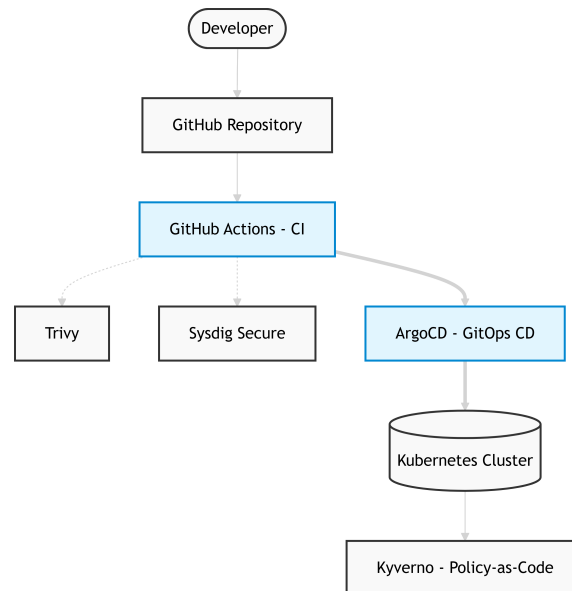


Figura 1.1: Arquitectura general del entorno DevSecOps

2 Marco teórico y análisis de referentes

2.1 Evolución del paradigma DevOps a DevSecOps

2.1.1 Limitaciones de los modelos tradicionales

Los modelos tradicionales de desarrollo de software, como el modelo en cascada, se caracterizan por una separación estricta de fases (análisis, desarrollo, pruebas y despliegue), lo que dificulta la detección temprana de errores y vulnerabilidades [1]. En estos enfoques, la seguridad se introduce habitualmente en etapas finales, incrementando significativamente el coste de corrección y el riesgo en producción.

La adopción de metodologías ágiles y, posteriormente, del paradigma DevOps permitió mejorar la colaboración entre equipos y acelerar los ciclos de entrega [2]. Sin embargo, diversos estudios destacan que la seguridad continuó tratándose como un proceso independiente, generando vulnerabilidades especialmente en entornos cloud-native [3].

En este contexto, se hace evidente la necesidad de integrar mecanismos de seguridad automatizados que puedan aplicarse de forma continua dentro del ciclo de vida del software.

2.1.2 Integración de la seguridad en el ciclo de vida del software

La evolución hacia DevSecOps implica la integración de la seguridad desde las primeras fases del desarrollo, incorporando controles automatizados en pipelines CI/CD [4]. Esto incluye análisis de vulnerabilidades, validación de configuraciones y control de cumplimiento.

La automatización permite garantizar consistencia y reducir errores humanos, especialmente en entornos dinámicos. Según [5], la integración temprana de la seguridad mejora significativamente la resiliencia del software.

Este enfoque resulta clave en el presente trabajo, donde la seguridad se implementa mediante mecanismos automatizados integrados en Kubernetes.

2.1.3 Nacimiento del concepto DevSecOps

DevSecOps surge como una evolución natural de DevOps, integrando la seguridad como un componente intrínseco del ciclo de vida del software [6]. Este paradigma promueve la automatización de controles de seguridad y la responsabilidad compartida entre equipos.

En entornos basados en contenedores y Kubernetes, este enfoque requiere mecanismos capaces de aplicar políticas de seguridad de forma dinámica y escalable.

En este trabajo, este objetivo se materializa mediante el uso de Policy-as-Code y herramientas como Kyverno, que permiten aplicar seguridad de forma declarativa y automatizada.

2.2 Kubernetes como plataforma de ejecución

2.2.1 Arquitectura de Kubernetes

Kubernetes se ha consolidado como el estándar de facto para la orquestación de contenedores, proporcionando una plataforma robusta para la gestión de aplicaciones distribuidas [7].

Su arquitectura se basa en un plano de control (control plane), que gestiona el estado del sistema, y nodos worker encargados de ejecutar las cargas de trabajo. El API Server actúa como punto central de interacción mediante un modelo declarativo [8].

Esta arquitectura permite automatizar la gestión del sistema, pero también introduce nuevos retos de seguridad que deben ser abordados mediante mecanismos específicos.

2.2.2 Componentes principales del clúster

Los componentes principales del control plane incluyen el API Server, el Scheduler y el Controller Manager, responsables de procesar solicitudes y mantener el estado deseado del sistema. En los nodos worker, el kubelet y el runtime de contenedores permiten la ejecución de los pods [8].

La existencia de múltiples componentes distribuidos hace necesario implementar controles de seguridad centralizados, como los Admission Controllers.

2.2.3 Gestión de red y Network Policies

Por defecto, Kubernetes permite la comunicación libre entre pods, lo que puede representar un riesgo significativo. Las Network Policies permiten definir reglas de comunicación basadas en etiquetas y namespaces, aplicando el principio de mínimo privilegio [8].

Además, en entornos reales es habitual encontrar configuraciones inseguras como:

- Contenedores ejecutándose como usuario root
- Escalada de privilegios habilitada
- Falta de límites de recursos
- Acceso no restringido a red

Estas prácticas suponen un riesgo elevado, especialmente en entornos multi-tenant.

Esto justifica la necesidad de aplicar controles automáticos en el momento del despliegue mediante políticas de seguridad en Kubernetes.

2.3 Policy-as-Code y Admission Controllers

2.3.1 Funcionamiento de los Admission Controllers

Los Admission Controllers son mecanismos del API Server que interceptan las solicitudes antes de que los recursos sean persistidos en el clúster [8]. Permiten aplicar validaciones o modificaciones sobre los objetos.

Este mecanismo constituye la base técnica sobre la que se implementará la seguridad en este trabajo.

2.3.2 Validación vs mutación de recursos

Los *validating admission controllers* permiten verificar que los recursos cumplen ciertas políticas, rechazando aquellos que no las satisfacen.

Por otro lado, los *mutating admission controllers* permiten modificar automáticamente los recursos antes de su creación, aplicando configuraciones seguras por defecto.

En este trabajo se dará especial importancia a las políticas de tipo *mutating*, ya que permiten aplicar mecanismos de *hardening* automático sobre los manifiestos, corrigiendo configuraciones inseguras sin intervención del usuario.

Este enfoque resulta clave para garantizar la seguridad en entornos DevSecOps altamente automatizados.

2.3.3 Importancia de la seguridad declarativa

El enfoque Policy-as-Code permite definir políticas de seguridad como código versionado, facilitando su integración en flujos GitOps [9].

Este modelo mejora la trazabilidad, reproducibilidad y automatización de la seguridad.

En este trabajo, estos mecanismos serán implementados mediante Kyverno para aplicar políticas automáticamente en tiempo de despliegue.

2.4 Herramientas y tecnologías utilizadas

2.4.1 Kyverno

Kyverno es una herramienta de Policy-as-Code diseñada específicamente para Kubernetes que permite definir políticas utilizando YAML, sin necesidad de aprender lenguajes adicionales [10].

A diferencia de otras soluciones, Kyverno permite definir políticas directamente en el mismo formato utilizado por Kubernetes, lo que facilita su adopción. Además, soporta tanto políticas de validación como de mutación, permitiendo no solo bloquear configuraciones inseguras, sino también corregirlas automáticamente.

Esta capacidad resulta especialmente relevante en entornos DevSecOps, donde la automatización y la consistencia son factores clave.

Kyverno constituye el eje central de este trabajo.

2.4.2 OPA Gatekeeper

Open Policy Agent (OPA) es un motor de políticas generalista que permite definir reglas mediante el lenguaje Rego [11]. Gatekeeper facilita su integración con Kubernetes.

Aunque ofrece gran flexibilidad, su complejidad y enfoque basado en validación lo diferencian de Kyverno.

2.4.3 ArgoCD y GitOps

ArgoCD implementa el paradigma GitOps, donde el estado del sistema se define en repositorios Git [12].

En este trabajo, se utilizará para automatizar el despliegue y garantizar la coherencia entre configuración y estado real.

2.4.4 GitHub Actions

GitHub Actions permite automatizar pipelines CI/CD, incluyendo procesos de build, test y seguridad [13].

Se utilizará para integrar controles de seguridad dentro del flujo DevSecOps.

2.4.5 Trivy y Sysdig Secure

Trivy permite el escaneo de vulnerabilidades en imágenes de contenedores [14], mientras que Sysdig Secure proporciona seguridad en tiempo de ejecución [15].

Estas herramientas complementan la seguridad aplicada mediante políticas en Kubernetes.

2.5 Comparativa teórica Kyverno vs OPA

Las herramientas Kyverno y Open Policy Agent (OPA) representan dos de las soluciones más relevantes dentro del enfoque *Policy-as-Code* aplicado a Kubernetes. Ambas permiten definir y aplicar políticas de seguridad sobre los recursos del clúster, aunque presentan diferencias significativas en cuanto a su diseño, enfoque y capacidades [10], [11].

Kyverno es una herramienta diseñada específicamente para Kubernetes (*Kubernetes-native*), que permite definir políticas utilizando el mismo formato YAML empleado en los manifiestos del propio sistema. Este enfoque reduce la complejidad y facilita su adopción por parte de equipos DevOps familiarizados con Kubernetes. Por el contrario, OPA utiliza el lenguaje Rego, un lenguaje declarativo de propósito general que ofrece una gran flexibilidad, pero

introduce una mayor curva de aprendizaje.

Desde el punto de vista funcional, ambas herramientas permiten implementar políticas de validación (*validating*), es decir, comprobar que los recursos cumplen determinadas condiciones antes de ser desplegados. Sin embargo, una diferencia clave radica en el soporte de políticas de mutación (*mutating*). Kyverno permite modificar automáticamente los recursos en tiempo de admisión, aplicando configuraciones seguras por defecto, mientras que OPA Gatekeeper se centra principalmente en la validación, careciendo de soporte nativo para la mutación de recursos.

A continuación, se presenta una comparativa resumida de ambas herramientas:

Característica	Kyverno	OPA Gatekeeper
Enfoque	Kubernetes-native	Generalista
Lenguaje de políticas	YAML	Rego
Curva de aprendizaje	Baja	Alta
Validación de recursos	Sí	Sí
Mutación de recursos	Sí	No (limitada)
Integración con Kubernetes	Nativa	Mediante CRDs
Facilidad de adopción	Alta	Media

Cuadro 2.1: Comparativa entre Kyverno y OPA Gatekeeper

Estas diferencias tienen un impacto directo en su aplicabilidad en entornos DevSecOps. Mientras que OPA ofrece una mayor expresividad y flexibilidad para definir políticas complejas, Kyverno destaca por su simplicidad operativa y su capacidad de integrarse de forma natural en flujos de trabajo basados en Kubernetes.

Desde la perspectiva de este trabajo, la capacidad de mutación de Kyverno resulta especialmente relevante, ya que permite implementar mecanismos de *hardening* automático sobre los manifiestos, corrigiendo configuraciones inseguras sin necesidad de intervención manual. Esto incluye, por ejemplo, la adición automática de límites de recursos, la desactivación de privilegios elevados o la aplicación de políticas de red por defecto.

En consecuencia, Kyverno se posiciona como una herramienta especialmente adecuada para entornos donde se prioriza la automatización, la consistencia y la integración con prácticas GitOps.

Esta comparativa teórica servirá como base para el análisis práctico que se desarrollará en capítulos posteriores, donde se evaluará el comportamiento de ambas soluciones en un entorno real de Kubernetes.

3 Objetivos y alcance

3.1 Objetivo general

El objetivo principal de este Trabajo de Fin de Máster es el diseño e implementación de un entorno funcional de DevSecOps sobre Kubernetes, integrando mecanismos de seguridad de forma nativa dentro del ciclo de vida del desarrollo y despliegue de aplicaciones.

El núcleo del proyecto se centra en la aplicación del paradigma Policy-as-Code mediante el uso de Admission Controllers, concretamente Kyverno, con el objetivo de garantizar configuraciones seguras en tiempo de despliegue. A diferencia de los enfoques tradicionales basados únicamente en la detección o el bloqueo, este trabajo pone especial énfasis en el uso de políticas de tipo *mutating*, capaces de corregir automáticamente configuraciones inseguras en los manifiestos de Kubernetes.

Esta aproximación permite no solo prevenir errores de configuración, sino también establecer mecanismos de hardening automático, alineados con las prácticas modernas de seguridad en entornos cloud-native.

3.2 Objetivos específicos

Para alcanzar el objetivo general, se definen los siguientes objetivos específicos:

- Desplegar un clúster Kubernetes funcional en un entorno de laboratorio, con una arquitectura basada en un nodo de control y un nodo worker, configurando adecuadamente su comunicación y la capa de red mediante una solución CNI como Calico.
- Instalar y configurar Kyverno como motor de políticas de seguridad basado en Policy-as-Code, integrándolo como Admission Controller dentro del clúster.
- Definir e implementar un conjunto representativo de políticas de seguridad (entre 5 y 10), centradas en problemáticas reales como:
 - ejecución de contenedores con privilegios elevados,
 - escalada de privilegios,
 - ausencia de límites de recursos,

- control de comunicaciones de red.
- Validar el comportamiento de las políticas mediante pruebas prácticas, demostrando tanto el bloqueo de configuraciones inseguras (*validating policies*) como su corrección automática (*mutating policies*).
- Implementar una política avanzada basada en la generación automática de Network Policies, con el objetivo de restringir accesos a recursos sensibles, como servicios de metadatos (por ejemplo, la IP 169.254.169.254), habitual en entornos cloud.
- Integrar el clúster dentro de un modelo de despliegue continuo basado en GitOps, utilizando ArgoCD para gestionar el estado declarativo de las aplicaciones.
- Incorporar un pipeline de integración continua mediante GitHub Actions, incluyendo procesos de construcción y análisis de vulnerabilidades con herramientas como Trivy o Sysdig Secure.
- Realizar una comparativa técnica entre Kyverno y OPA Gatekeeper, analizando diferencias en facilidad de uso, capacidades de validación y mutación, y adecuación en entornos productivos.

3.3 Alcance del proyecto

El proyecto se desarrolla en un entorno de laboratorio basado en máquinas virtuales, con el objetivo de reproducir un escenario realista de despliegue en Kubernetes. El alcance incluye la configuración del clúster, la integración de las herramientas principales del pipeline DevSecOps y la implementación de políticas de seguridad representativas.

No obstante, el trabajo no pretende cubrir de manera exhaustiva todos los estándares de seguridad existentes, sino demostrar el funcionamiento y la viabilidad del modelo Policy-as-Code aplicado a Kubernetes mediante casos prácticos.

Asimismo, la utilización de herramientas de automatización como Ansible se considera una posible ampliación del proyecto. Siguiendo un enfoque iterativo, se prioriza inicialmente la implementación de los mecanismos de seguridad y su validación, dejando la automatización completa del despliegue como una línea de trabajo futura o paralela en función del tiempo disponible.

Bibliografía

- [1] R. S. Pressman y B. R. Maxim, *Software Engineering: A Practitioner's Approach*. McGraw-Hill, 2019.
- [2] G. Kim, J. Humble, P. Debois y J. Willis, "The DevOps Handbook," 2016.
- [3] T. Sharma et al., "DevSecOps: A Multivocal Literature Review," 2020. dirección: <https://arxiv.org/abs/2007.12561>
- [4] A. Behl y K. Behl, "DevSecOps: Integrating Security into DevOps," 2017.
- [5] A. Rahman y L. Williams, "DevSecOps: Integrating Security into Continuous Delivery," 2019.
- [6] V. Mohan y L. Ben Othmane, "Towards DevSecOps: Integrating Security into DevOps," 2016. dirección: <https://ieeexplore.ieee.org/document/7488962>
- [7] B. Burns, B. Grant, D. Oppenheimer, E. Brewer y J. Wilkes, "Borg, Omega, and Kubernetes," *Communications of the ACM*, 2016. dirección: <https://dl.acm.org/doi/10.1145/2934556.2934564>
- [8] The Kubernetes Authors, *Kubernetes Documentation*, 2024. dirección: <https://kubernetes.io/docs/home/>
- [9] M. Hüttermann, *DevOps for Developers*. 2022.
- [10] Kyverno Project, *Kyverno Documentation*, 2024. dirección: <https://kyverno.io/>
- [11] OPA Project, *Open Policy Agent Documentation*, 2024. dirección: <https://www.openpolicyagent.org/>
- [12] Weaveworks, *GitOps Explained*, 2021. dirección: <https://www.weave.works/technologies/gitops/>
- [13] GitHub, *GitHub Actions Documentation*, 2024. dirección: <https://docs.github.com/en/actions>
- [14] Aqua Security, *Trivy Documentation*, 2024. dirección: <https://aquasecurity.github.io/trivy/>
- [15] Sysdig, *Sysdig Secure*, 2024. dirección: <https://sysdig.com/products/secure/>